

Задача

Разработать веб-сервис расчета IRR и механизм тестирования разработанного сервиса в автоматическом режиме по расписанию в выбранной ветке

Порядок выполнения

Шаг 1. Установите все ПО из раздела «обеспечение». Сделайте проект в GIT и отдельную ветку для разработки. Разработку необходимо вести в выделенной ветке

Visual Studio Code

```
Flask      3.1.3
numpy      2.2.6
numpy-financial 1.0.0
```

Шаг 2. Реализация ручного расчёта IRR

IRR — это ставка r , при которой NPV (чистая приведённая стоимость) равна нулю:

$$NPV(r) = \sum_{t=0..n} CF_t / (1 + r)^t = 0$$

Аналитического решения для $n > 1$ не существует, поэтому используем численный метод — бисекцию (метод деления отрезка пополам).

Для вычисления значения используется библиотека `numpy-financial`

Код `irr_calc.py`

```
import numpy_financial as npf

def irr_calculation(values):
    result = npf.irr(values)

    if result is None:
        raise ValueError("IRR could not be calculated")
    return float(result)
```

Шаг 3. Flask-приложение, которое в виде веб-сервиса возвращает значение IRR по заданным параметрам для расчета

Код flask_service.py

```
from flask import Flask, request, jsonify

from irr_calc import irr_calculation

app = Flask(__name__)

@app.route("/irr", methods=["POST"])
def calculate_irr():
    try:
        data = request.get_json()

        if not data or "values" not in data:
            return jsonify({"error": "You must specify values"}), 400

        values = data["values"]

        if not isinstance(values, list):
            return jsonify({"error": "values must be a list"}), 400

        result = irr_calculation(values)

        return jsonify({"irr": result}), 200

    except Exception as error:
        return jsonify({"error": str(error)}), 400

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "ok"}), 200

if __name__ == '__main__':
    app.run(host="127.0.0.1", port=5000, debug=False)
```

```
(venv) PS C:\mirea\devops\irr_project_1> python .\flask_service.py
* Serving Flask app 'flask_service'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [08/Oct/2026 20:31:47] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 20:31:47] "GET /favicon.ico HTTP/1.1" 404 -
```

Результат расчета для значений [-1000, 300, 400, 500]

```
(venv) PS C:\mirea\devops\irr_project_1> curl.exe http://127.0.0.1:5000/health
{"status":"ok"}
(venv) PS C:\mirea\devops\irr_project_1> python -c "import requests; s=requests.Session(); s.trust_env=False; r=s.post('http://127.0.0.1:5000/irr', json={'values':[-1000,300,400,500]}); print(r.status_code); print(r.json())"
200
{'irr': 0.08896339469335035}
(venv) PS C:\mirea\devops\irr_project_1>
```

Шаг 4. Unit-тесты. Разработайте приложение, которое вызывает приложение шаг 2 (вызов выполняется HTTP-запросом к Flask-сервису, например, через requests) и в txt файл записывает результаты вызова (код возврата, результат, время выполнения в мс). В приложении должен быть набор эталонных данных для 3ех эталонных вызовов и 10 вызов с данными Random в широких границах

код unit_tests.py

```
import random
import time
from datetime import datetime

import requests
import numpy_financial as npf

URL = "http://127.0.0.1:5000/irr"
RESULT_FILE = "unit_test_results.txt"

session = requests.Session()
session.trust_env = False

REFERENCE_CASES = [
    [-1000, 300, 400, 500],
    [-10000, 2000, 3000, 4000, 5000],
    [-5000, 1000, 1500, 2000, 2500]
]

def random_cash_flows():
    n = random.randint(3, 10)
    values = [-random.randint(1000, 100000)]

    for _ in range(n - 1):
        values.append(random.randint(100, 100000))

    return values
```

```

def test(name, values):
    start = time.perf_counter()

    try:
        response = session.post(
            URL,
            json={"values": values},
            timeout=10
        )

        time_ms = (time.perf_counter() - start) * 1000

        data = response.json()
        actual = data.get("irr")
        expected = npf.irr(values)

        passed = (
            response.status_code == 200
            and actual is not None
            and expected is not None
            and abs(actual - expected) < 1e-5
        )

        return (
            f"{name}\n"
            f"Values: {values}\n"
            f"HTTP: {response.status_code}\n"
            f"IRR: {actual}\n"
            f"Expected: {expected}\n"
            f"Time: {time_ms:.3f} ms\n"
            f"PASSED: {passed}\n"
        )

    except Exception as error:
        time_ms = (time.perf_counter() - start) * 1000

        return (
            f"{name}\n"
            f"Values: {values}\n"
            f"ERROR: {error}\n"
            f"Time: {time_ms:.3f} ms\n"

```

```

        f"PASSED: False\n"
    )

def main():
    results = []

    for i, values in enumerate(REFERENCE_CASES, 1):
        results.append(test(f"Reference {i}", values))

    for i in range(1, 11):
        values = random_cash_flows()
        results.append(test(f"Random {i}", values))

    with open(RESULT_FILE, "a", encoding="utf-8") as file:
        file.write("\n" + "=" * 50 + "\n")
        file.write(
            "TEST RUN: "
            + datetime.now().strftime("%Y-%m-%d %H:%M:%S")
            + "\n"
        )
        file.write("=" * 50 + "\n")

        for result in results:
            file.write(result + "\n")

        passed = sum("PASSED: True" in result for result in results)
        file.write(f"SUMMARY: {passed}/{len(results)} tests passed\n")

if __name__ == "__main__":
    main()

```

Результат unit_test_result.txt

```
1
2  =====
3  TEST RUN: 2026-10-08 23:36:57
4  =====
5  Reference 1
6  Values: [-1000, 300, 400, 500]
7  HTTP: 200
8  IRR: 0.08896339469335035
9  Expected: 0.08896339469335035
10 Time: 21.828 ms
11 PASSED: True
12
13 Reference 2
14 Values: [-10000, 2000, 3000, 4000, 5000]
15 HTTP: 200
16 IRR: 0.1282572690016739
17 Expected: 0.1282572690016739
18 Time: 15.690 ms
19 PASSED: True
20
21 Reference 3
22 Values: [-5000, 1000, 1500, 2000, 2500]
23 HTTP: 200
24 IRR: 0.1282572690016739
25 Expected: 0.1282572690016739
26 Time: 15.279 ms
27 PASSED: True
28
```

```
29 Random 1
30 Values: [-4322, 96559, 9500, 64958, 89973, 196, 22487, 78755]
31 HTTP: 200
32 IRR: 21.470697567210962
33 Expected: 21.470697567210962
34 Time: 15.883 ms
35 PASSED: True
36
37 Random 2
38 Values: [-2156, 81859, 69499, 95242, 28120, 32948, 3315, 79179, 46923]
39 HTTP: 200
40 IRR: 37.82773727587751
41 Expected: 37.82773727587751
42 Time: 15.307 ms
43 PASSED: True
44
45 Random 3
46 Values: [-15965, 11228, 53160, 43710, 39656, 47035, 44317, 42055]
47 HTTP: 200
48 IRR: 1.6125645729780222
49 Expected: 1.6125645729780222
50 Time: 15.244 ms
51 PASSED: True
52
53 Random 4
54 Values: [-32759, 12600, 3705, 10040, 29634, 25138, 79069, 93303]
55 HTTP: 200
56 IRR: 0.5203638625625919
57 Expected: 0.5203638625625919
58 Time: 15.363 ms
59 PASSED: True
```

Шаг 5. Сделайте приложение, которое обновляет репозиторий из ветки Main и запускает приложение Шаг 4. Если приложение уже запущено, то его необходимо остановить

```
(venv) PS C:\mirea\devops\irr_project_1> git switch -c main
Switched to a new branch 'main'
(venv) PS C:\mirea\devops\irr_project_1> git branch
dev
* main
```

Код updater.ps1

```
# Путь к проекту
$ProjectPath = "C:\mirea\devops\irr_project_1"
```

```

Set-Location $ProjectPath

# Переключаемся на main
git switch main
if ($LASTEXITCODE -ne 0) { exit 1 }

# Python и файлы проекта
$python = "$ProjectPath\venv\Scripts\python.exe"
$serviceFile = "$ProjectPath\flask_service.py"
$testFile = "$ProjectPath\unit_tests.py"

# Останавливаем процесс на порту 5000
Get-NetTCPConnection -LocalPort 5000 -State Listen -ErrorAction SilentlyContinue |
    ForEach-Object { Stop-Process -Id $_.OwningProcess -Force }

# Запускаем Flask
Start-Process -FilePath $python -ArgumentList $serviceFile -WorkingDirectory $ProjectPath

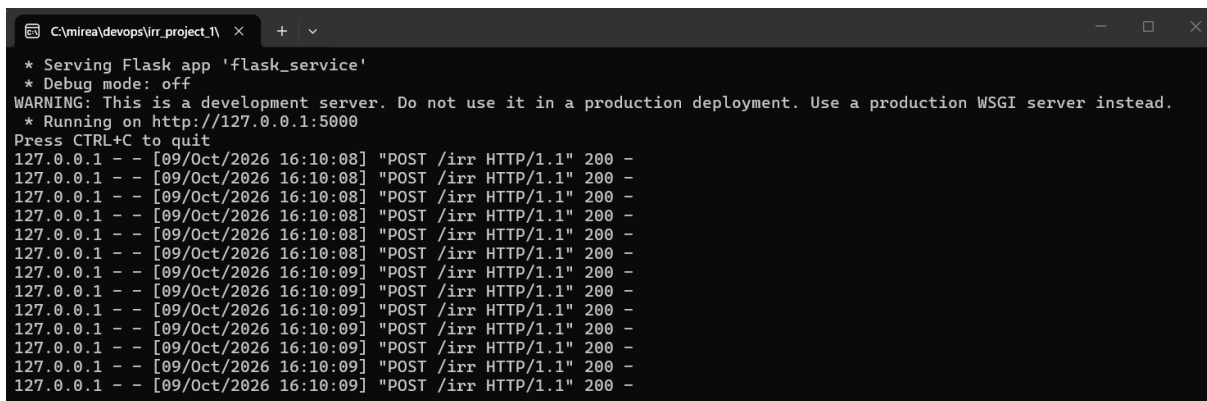
# Ждём запуска сервера
Start-Sleep -Seconds 3

# Проверяем сервер
Invoke-RestMethod "http://127.0.0.1:5000/health"

# Запускаем тесты
& $python $testFile

```

Запуск скрипта в PowerShell



```

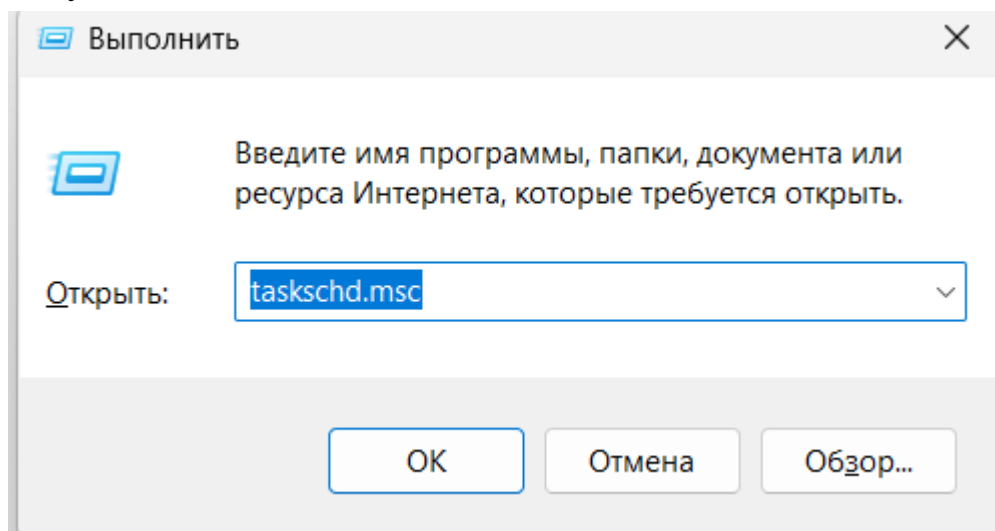
C:\mirea\devops\irr_project\1\ x + v
* Serving Flask app 'flask_service'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [09/Oct/2026 16:10:08] "POST /irr HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 16:10:08] "POST /irr HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 16:10:08] "POST /irr HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 16:10:08] "POST /irr HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 16:10:08] "POST /irr HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 16:10:08] "POST /irr HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 16:10:08] "POST /irr HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 16:10:09] "POST /irr HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 16:10:09] "POST /irr HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 16:10:09] "POST /irr HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 16:10:09] "POST /irr HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 16:10:09] "POST /irr HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 16:10:09] "POST /irr HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 16:10:09] "POST /irr HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 16:10:09] "POST /irr HTTP/1.1" 200 -

```

Запись новых результатов в txt файл


```
220 =====
221 TEST RUN: 2026-10-09 16:10:09
222 =====
223 Reference 1
224 Values: [-1000, 300, 400, 500]
225 HTTP: 200
226 IRR: 0.08896339469335035
227 Expected: 0.08896339469335035
228 Time: 35.407 ms
229 PASSED: True
230
231 Reference 2
232 Values: [-10000, 2000, 3000, 4000, 5000]
233 HTTP: 200
234 IRR: 0.1282572690016739
235 Expected: 0.1282572690016739
236 Time: 26.415 ms
237 PASSED: True
```

Шаг 6. Запустите приложение Шаг 5 через Cron на работу каждые 2 минуты



Создание триггера



Начать задачу: По расписанию

Настройки

- ☐ Однократно
- ☒ Ежедневно
- ☐ Еженедельно
- ☐ Ежемесячно

Начать: 09.10.2026 16:23:10 ☐ Синхр. по поясам

Повторять каждые: 1 дн.

Дополнительные параметры

☐ Отложить задачу на (произвольная задержка): 1 ч.

☒ Повторять задачу каждые: 2 мин. в течение: Бесконечно

☒ Останавливать все задачи по истечении срока повторов

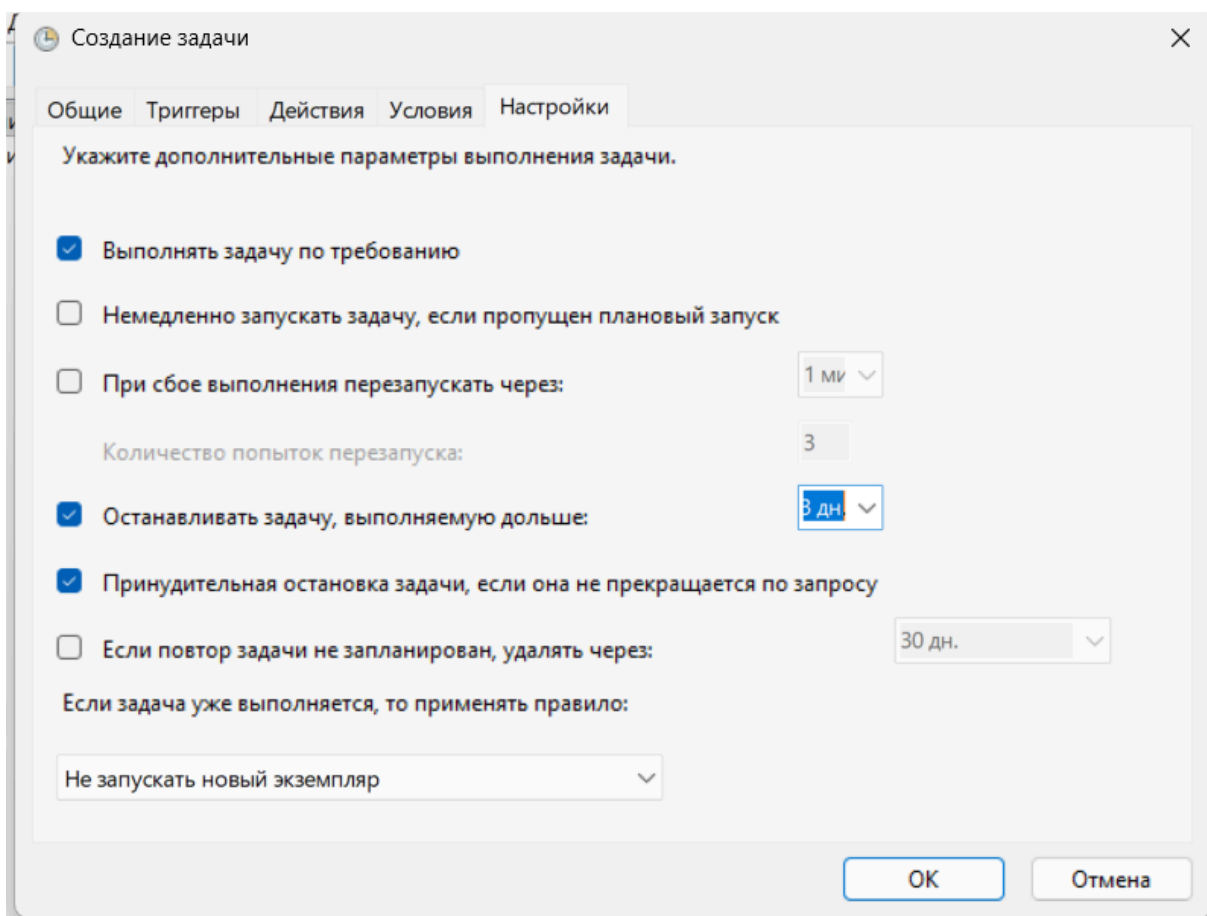
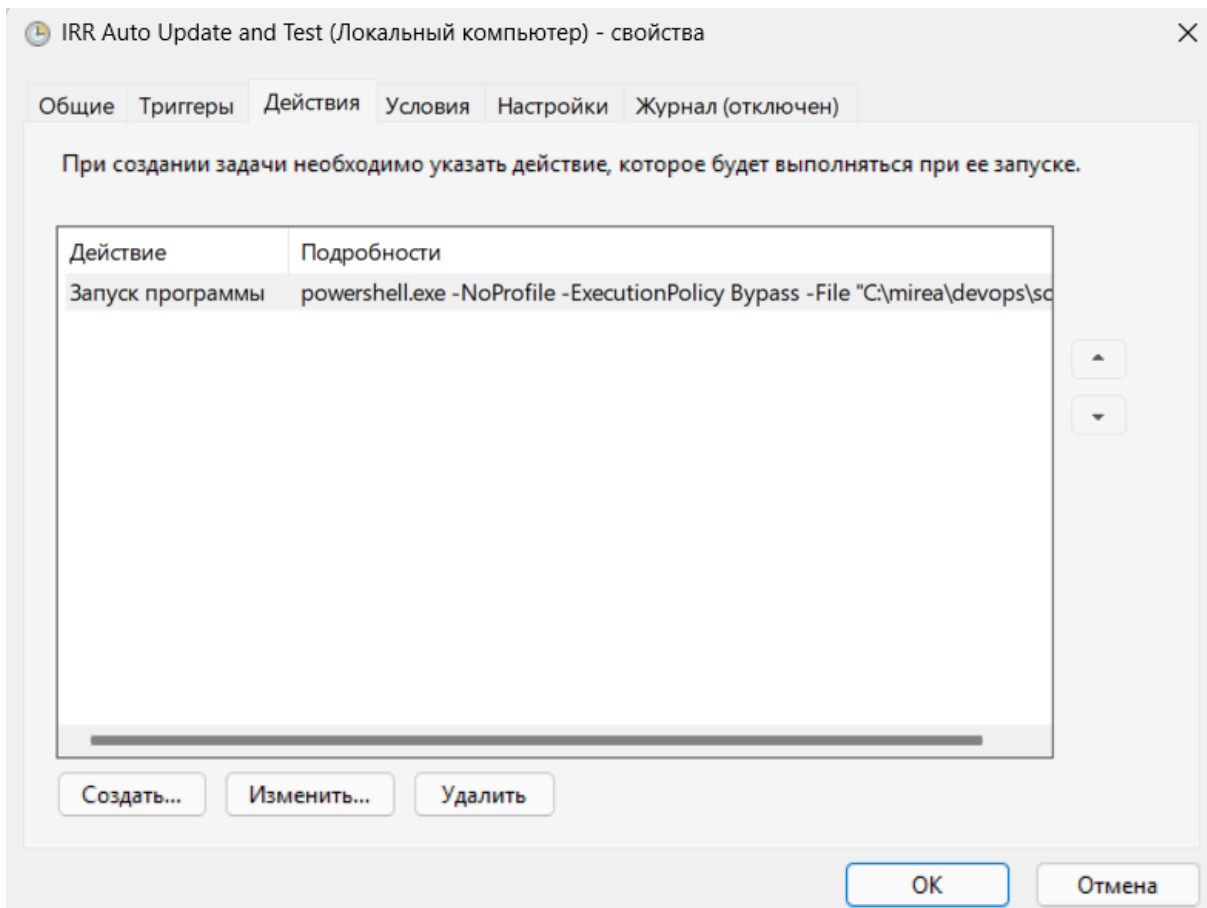
☐ Остановить задачу через: 3 дн.

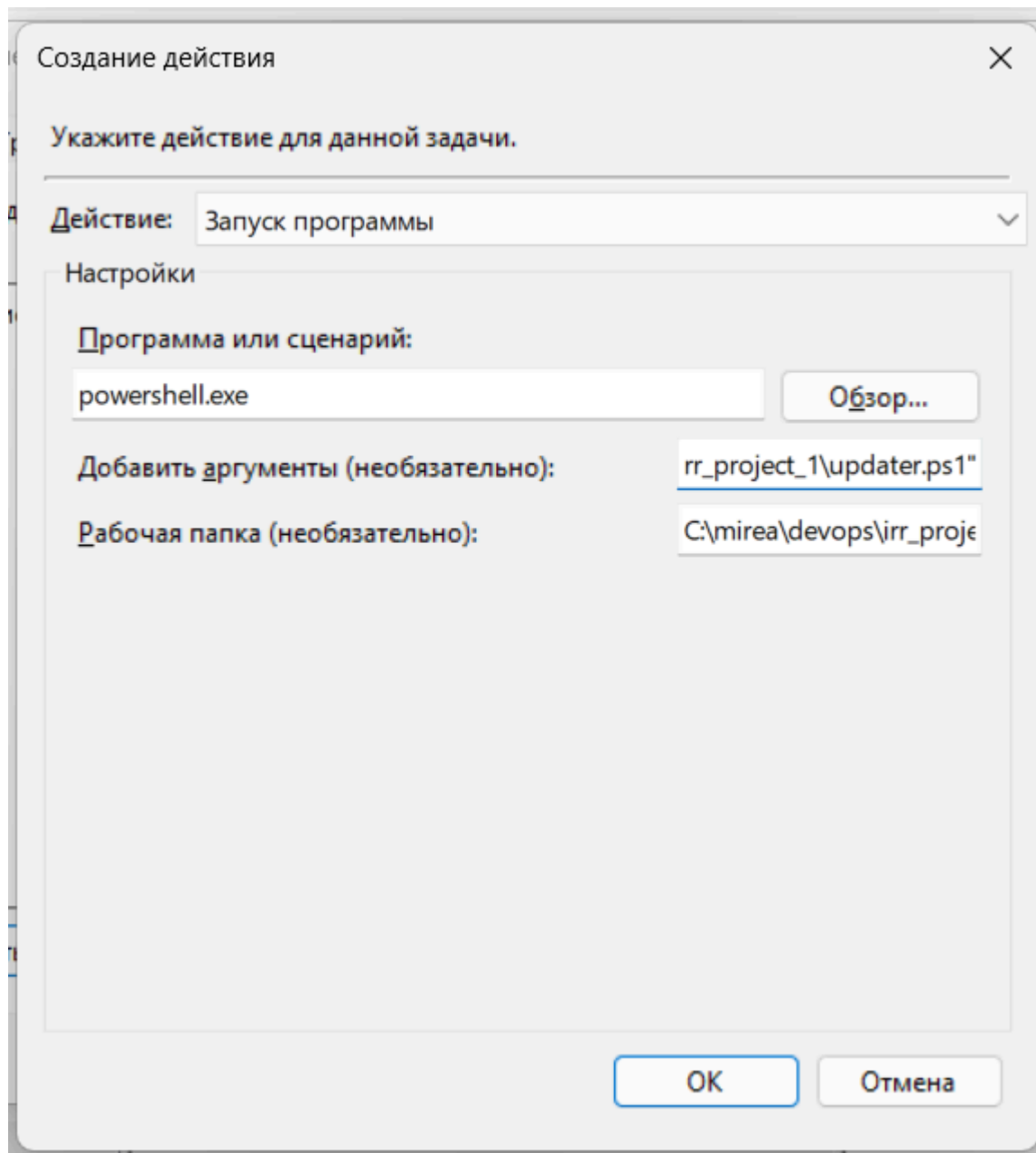
☐ Срок действия: 09.10.2027 16:23:13 ☐ Синхр. по поясам

☒ Включено

OK

Отмена





Запуск каждые 2 минуты

```
C:\mirea\devops\irr_project_1\ >
* Serving Flask app 'flask_service'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [09/Oct/2026 16:53:17] "POST /irr HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 16:53:17] "POST /irr HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 16:53:17] "POST /irr HTTP/1.1" 200 -
```

Шаг 7. Сделайте merge ветки разработки в main. Убедитесь, что cron подхватил изменения. Оцените результаты работы приложения Шаг 6.

```
(venv) PS C:\mirea\devops\irr_project_1> git merge dev
Merge made by the 'ort' strategy.
 unit_tests.py | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
(venv) PS C:\mirea\devops\irr_project_1> git log -1 --oneline
da2a99e (HEAD -> main) Merge branch 'dev'
```

```
1637 =====
1638 TEST RUN: 2026-10-09 19:09:18
1639 =====
1640 Reference 1
1641 Values: [-1000, 300, 400, 500]
1642 HTTP: 200
1643 IRR: 0.08896339469335035
1644 Expected: 0.08896339469335035
1645 Time: 22.172 ms
1646 PASSED: True
1647
1648 Reference 2
1649 Values: [-10000, 2000, 3000, 4000, 5000]
1650 HTTP: 200
1651 IRR: 0.1282572690016739
1652 Expected: 0.1282572690016739
1653 Time: 29.697 ms
1654 PASSED: True
1655
1656 Reference 3
1657 Values: [-5000, 1000, 1500, 2000, 2500]
1658 HTTP: 200
1659 IRR: 0.1282572690016739
1660 Expected: 0.1282572690016739
1661 Time: 6.785 ms
1662 PASSED: True
1663
```

```
1746 =====
1747 TEST RUN: 2026-10-09 19:11:18
1748 =====
1749 Reference 1
1750 Values: [-1000, 300, 400, 500]
1751 HTTP: 200
1752 IRR: 0.08896339469335035
1753 Expected: 0.08896339469335035
1754 Time: 6.572 ms
1755 PASSED: True
1756
1757 Reference 2
1758 Values: [-10000, 2000, 3000, 4000, 5000]
1759 HTTP: 200
1760 IRR: 0.1282572690016739
1761 Expected: 0.1282572690016739
1762 Time: 23.069 ms
1763 PASSED: True
1764
1765 Reference 3
1766 Values: [-5000, 1000, 1500, 2000, 2500]
1767 HTTP: 200
1768 IRR: 0.1282572690016739
1769 Expected: 0.1282572690016739
1770 Time: 14.928 ms
1771 PASSED: True
1772
1773 Random 1
```

Шаг 8. Внесите ошибку в приложение расчета IRR. Сделайте merge ветки разработки в main. Оцените результат работы приложения Шаг 6.

в функции расчета irr добавлена ошибка, к результату прибавляется 0.1

```
return float(result) + 0.1
```

```
1855  =====
1856  TEST RUN: 2026-10-09 19:24:25
1857  =====
1858  Reference 1
1859  Values: [-1000, 300, 400, 500]
1860  HTTP: 200
1861  IRR: 0.18896339469335036
1862  Expected: 0.08896339469335035
1863  Time: 26.412 ms
1864  PASSED: False
1865
1866  Reference 2
1867  Values: [-10000, 2000, 3000, 4000, 5000]
1868  HTTP: 200
1869  IRR: 0.2282572690016739
1870  Expected: 0.1282572690016739
1871  Time: 28.839 ms
1872  PASSED: False
1873
```